

COURS ILLUSTRÉ

Comprendre et construire Lmaana Assistant

De la voix en Darija à une réponse documentée

Un cours de bout en bout pour comprendre les modèles, les données, le code et les décisions de qualité derrière ton projet. Chaque notion est reliée à un problème concret : reconnaître une phrase, retrouver une preuve, conserver une condition ou expliquer un refus de réponse.

01**02****03****Reconnaître**

Audio -> texte
Lmaana 2.4 · prévu

**Retrouver**

Question -> preuves
Recherche · active

**Répondre**

Preuves -> rédaction
Qwen · qualité à évaluer

Figure 01. Trois capacités distinctes : reconnaître, retrouver et répondre. Aucune ne garantit à elle seule les deux autres.

Ce que tu sauras faire

Expliquer le rôle de Lmaana 2.4, Qwen3, Ollama, Qdrant, FastAPI et Streamlit ; calculer des métriques simples ; diagnostiquer une mauvaise réponse ; lire les principaux modules ; concevoir des tests qui séparent un logiciel fonctionnel d'un assistant réellement fiable.

Niveau : débutant motivé à intermédiaire. Prérequis : variables, fonctions et listes Python ; notions élémentaires de terminal. Les formules sont expliquées. Les exemples chiffrés fictifs sont signalés.

Photographie du projet : 3 octobre 2026. Le chemin texte fonctionne avec un modèle local. L'entrée vocale Lmaana 2.4 et la synthèse vocale ne sont pas intégrées. La qualité de génération en Darija reste à améliorer. Ce cours n'est pas un guide administratif ni une certification de réponses.

Établi à partir du code local, du README, des rapports de vérification et des références primaires listées en fin de cours.

MODE D'EMPLOI

Comment étudier ce cours

Comprendre les responsabilités avant de mémoriser les noms

Lis d'abord les chapitres 01 à 05 pour construire une carte mentale. Poursuis avec la voix et les langues, puis la préparation des sources et la recherche. Les derniers chapitres transforment ces composants en service testable. Ne cherche pas à retenir tous les paramètres dès la première lecture : demande-toi toujours quelle erreur chaque mécanisme évite.

Parcours	Chapitres	Question directrice
Fondations	01 - 05	Quel composant fait quel travail ?
Voix et langues	06 - 09	Comment conserver ce que l'utilisateur veut dire ?
Connaissance et recherche	10 - 16	Quelle preuve peut réellement soutenir la réponse ?
Réponse et logiciel	17 - 24	Comment servir et contrôler le résultat ?
Évaluation et pratique	25 - 31	Comment savoir si une modification améliore le projet ?
Révision	Annexes	Exercices, corrigés, glossaire et références.

Trois statuts à ne jamais mélanger

Actif désigne un chemin exécuté sur cette machine. **Adaptateur présent** indique que du code et des tests de contrat existent, sans démonstration complète avec le vrai modèle. **Prévu** décrit la cible future. Un dessin d'architecture peut contenir ces trois statuts ; il doit les rendre visibles.

La méthode de travail

Après chaque chapitre, explique la notion à voix haute avec un exemple différent. Puis ouvre le fichier indiqué et retrouve la responsabilité correspondante. Enfin, imagine un contre-exemple : une langue ambiguë, un PDF tronqué, un modèle indisponible. Si tu sais prédire la réaction du système, tu as compris davantage qu'une simple définition.

Les durées d'apprentissage dépendent de ton expérience. Travaille par petites séances : une notion, un calcul, un test. Les exercices finaux utilisent des données fictives pour éviter de confondre apprentissage technique et conseil administratif.

01 · FONDATIONS

L'architecture de bout en bout

Le produit est un système, pas un modèle unique

Lmaana Assistant assemble plusieurs programmes spécialisés. Streamlit reçoit la question et affiche le résultat. FastAPI vérifie la requête et appelle un pipeline Python. Le pipeline prépare le texte, retrouve des passages dans Qdrant, filtre les sources, puis demande à Qwen de rédiger via Ollama. Des contrôles décident si cette rédaction peut être affichée.

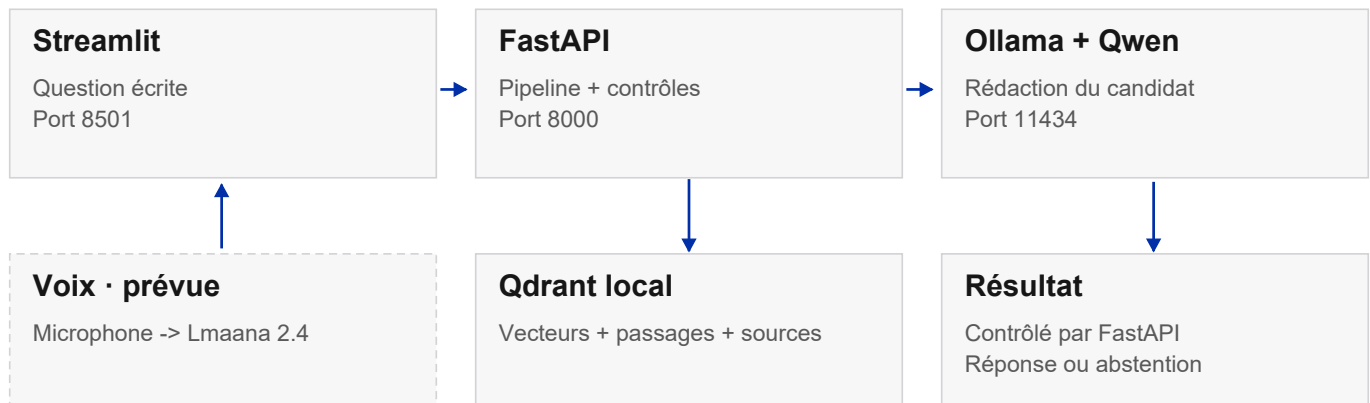


Figure 02. Le trajet actif est textuel. La voix est une extension prévue, pas une fonctionnalité déjà disponible.

La **séparation des responsabilités** permet de remplacer un composant sans réécrire tout le produit. Une mauvaise transcription relève de l'ASR ; un document introuvable relève de la recherche ; un chiffre inventé relève de la génération ou de ses contrôles. Sans cette séparation, chaque échec devient un vague « problème d'IA ».

L'**ingestion** prépare les documents avant les questions. L'**inférence** utilise les modèles lorsqu'une requête arrive. Ces deux chemins n'ont pas le même rythme : on n'extrait pas tous les PDF à chaque question, et on n'entraîne pas Qwen lorsqu'un utilisateur clique sur Envoyer.

Actuellement : recherche lexicale hachée + Qdrant local + Qwen3-4B via Ollama. Qwen3-Embedding a un adaptateur optionnel non validé en exécution réelle ici. LangChain, LangGraph, FAISS et React étaient des options évoquées, pas la pile implémentée.

À lire : `src/lmaana_assistant/pipeline.py` ; `api/app.py` ; `apps/streamlit_app.py`.

02 · FONDATIONS

Ce qu’est un modèle de langage

Des poids appris et une prédiction de séquence

Un modèle est une fonction paramétrée. Ses **poids** sont des nombres ajustés pendant l’entraînement. Dans un modèle de langage causal, la fonction attribue des probabilités aux tokens suivants à partir des tokens déjà présents. Un token peut correspondre à un mot, une partie de mot, un signe ou une portion d’octets selon le tokenizer.

$$P(\text{réponse} \mid \text{contexte}) = \text{produit des } P(\text{token_t} \mid \text{contexte, tokens précédents})$$

Cette règle très simple, répétée sur des réseaux et des données considérables, permet des comportements complexes : reformulation, traduction, raisonnement partiel, rédaction. Elle ne donne pas au modèle une base de faits garantie. Une continuation plausible peut être fausse, notamment si les données sont anciennes ou si une condition est perdue.

L’idée du Transformer

L’attention combine des représentations de positions différentes pour construire le contexte utile à chaque token. Dans un décodeur causal, une position ne doit pas voir les tokens futurs. Des couches successives transforment ces représentations ; la sortie finale sert à choisir le prochain token. Attention n’est donc pas synonyme de vérification des sources. [R2]

Dans « les banques partenaires », le mot « partenaires » modifie une restriction essentielle. Un système peut écrire une phrase grammaticalement parfaite en supprimant cette restriction. Voilà pourquoi la qualité linguistique, la compréhension des conditions et la vérité factuelle doivent être évaluées séparément.

Trois notions pratiques

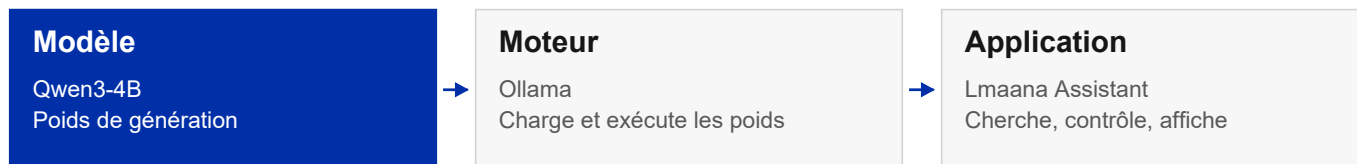
Notion	Signification
Préentraînement	Apprendre de larges régularités de langue et de données.
Post-entraînement	Adapter le comportement, par exemple au suivi d’instructions.
Inférence	Utiliser les poids existants pour produire une sortie.

Qwen3-4B est le modèle génératif retenu. Le nom « 4B » indique un ordre de grandeur de quatre milliards de paramètres, pas quatre milliards de documents ni une taille de mémoire fixe. [R3]

03 · FONDATIONS

Ollama, Qwen et Lmaana

Un moteur, un modèle de texte et un modèle de parole



Ne pas confondre identité du modèle et logiciel d'exécution.

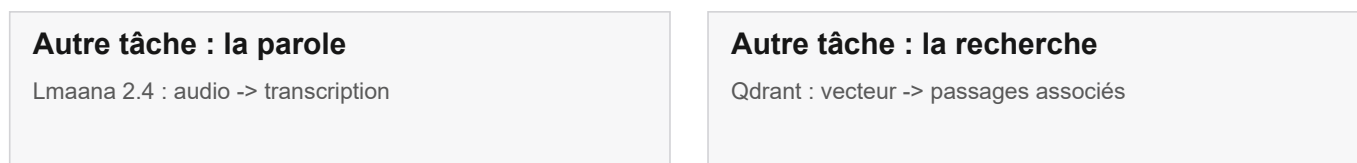


Figure 03. Ces noms appartiennent à des niveaux différents. Changer de moteur ne garantit pas une meilleure compétence linguistique.

Ollama est le service d'exécution : il gère un modèle local et expose une API. **Qwen3-4B** représente les poids utilisés pour générer du texte. **Lmaana 2.4** est la cible de reconnaissance vocale, avec un autre runtime. Ollama n'est donc ni le cerveau complet du RAG, ni la transcription Lmaana, ni le moteur de recherche documentaire.

Le service local est interrogé sur le port 11434. L'application utilise `/api/tags` et `/api/show` pour contrôler la présence et les métadonnées du modèle, puis `/api/chat` pour la génération. Le paramètre de format permet de demander une sortie structurée. Ces appels ne remplacent pas les contrôles métier du projet. [R4, R5]

Nom, version et identité

Un **tag** comme `qwen3:4b` est pratique mais peut désigner un artefact différent après une mise à jour. Un **digest** identifie précisément un manifeste. Épingler ce digest évite un changement silencieux. Un GGUF importé et un modèle du registre peuvent différer par les poids, les métadonnées ou le template de conversation : leurs résultats doivent être comparés, pas supposés identiques.

Dans ce projet, Ollama a effectivement exécuté Qwen sur GPU. L'adaptateur `llama.cpp` autonome existe, mais son exécution locale reste non validée. L'installation d'un runtime ne prouve ni la fidélité de ses réponses ni sa maîtrise de la Darija.

À lire : `generation/ollama.py` ; `generation/adapters.py` ; `configs/Modelfile.qwen3`.

04 · FONDATIONS

Tokens, contexte et décodage

La place disponible et la manière de choisir les mots

La **fenêtre de contexte** limite la séquence manipulée pendant une génération. Le prompt comprend les instructions, la question, les passages et le formatage de conversation. Il faut aussi réserver de la place à la réponse. Une limite exprimée en tokens n'est pas un nombre de mots constant : le français, l'arabe et l'Arabizi ne se découpent pas nécessairement de la même façon.

instructions + question + preuves + template + sortie <= fenêtre de contexte

Le profil local configuré réserve 8 192 tokens de contexte et jusqu'à 1 024 tokens de sortie. L'adaptateur Ollama estime prudemment la taille à partir des octets UTF-8 avec une réserve de template. Ce n'est pas un comptage exact. Si nécessaire, il retire des passages entiers de moindre rang ; il ne doit pas tronquer silencieusement une condition au milieu d'une phrase.

Les paramètres de génération

Paramètre	Intuition	Attention
Température	Modifie la concentration des probabilités.	Plus faible ne signifie pas plus vrai.
top-k	Garde les k candidats les plus probables.	Un k trop petit réduit les possibilités.
top-p	Garde un ensemble couvrant une masse de probabilité.	Il dépend de la distribution courante.
Limite de sortie	Arrête une réponse devenue trop longue.	Une sortie coupée peut être invalide.

Les réglages actifs sont température 0,7, top-p 0,8 et top-k 20. Ils sont des choix de configuration, pas des garanties. Le projet demande think=false et stream=false : pas de raisonnement séparé demandé, et récupération de la réponse complète avant validation. Un mode de réflexion activé n'a pas résolu les défauts linguistiques observés dans l'essai diagnostique local.

Le modèle ne reçoit pas automatiquement tout l'historique de l'interface. Une conversation mémorisée exige une politique explicite : quelles anciennes questions transmettre, pendant combien de temps, et avec quel budget ? Le chemin actuel traite chaque question avec ses preuves.

À lire : generation/ollama.py. Références : [R3, R4].

05 · FONDATIONS

RAM, VRAM et quantification

Pourquoi « le fichier tient » ne suffit pas

La **RAM** est la mémoire principale du système ; la **VRAM** est la mémoire du GPU. La machine de ce projet possède environ 32 Go de RAM et un GPU RTX 5070 Laptop avec environ 8 Go de VRAM. Ce sont deux budgets distincts. Un modèle peut tenir dans la RAM sans tenir entièrement sur le GPU.

Sur disque

Fichier de poids

Q4_K_M : valeurs quantifiées
+ échelles + métadonnées
Environ 2,5 Go dans ce projet



Pendant l'inférence

Poids chargés

Cache KV : contexte et séquences

Activations, buffers, affichage

RAM système ~32 Go | VRAM GPU ~8 Go | Schéma non proportionnel

Figure 04. Budget conceptuel de mémoire, non à l'échelle. Les activations et le cache s'ajoutent toujours aux poids.

La **quantification** représente les poids avec moins de bits. À titre d'ordre de grandeur : quatre milliards de paramètres à 4 bits donnent environ 2 milliards d'octets bruts. Il faut ajouter échelles, métadonnées et parties stockées différemment. Le GGUF Q4_K_M officiel téléchargé occupe environ 2,5 Go ; cela ne prédit pas à lui seul la mémoire totale d'exécution. [R3]

mémoire des poids, approximation = paramètres × bits / 8

Le **cache KV** conserve des représentations utiles à l'attention pour les tokens déjà traités. À architecture et précision fixées, il augmente avec la longueur du contexte et le nombre de séquences concurrentes. S'ajoutent les activations temporaires, les buffers du runtime et la mémoire utilisée par l'affichage.

L'**offload** répartit le calcul entre CPU et GPU. Il peut permettre un modèle plus gros, mais les transferts et le CPU peuvent ralentir la réponse. La bonne stratégie est de mesurer chaque modèle seul, puis leur coexistence. Charger simultanément ASR et LLM n'est pas une promesse raisonnable sur la seule base des tailles de fichiers.

Le relevé précédent d'Ollama indiquait environ 3,9 Go chargés pour Qwen et un placement 100 % GPU. C'est une observation datée pour une configuration, pas une garantie de consommation sous toute charge.

06 · VOIX ET LANGUES

De l'onde sonore au texte

ASR, prétraitement audio et erreurs en cascade

L'**ASR**, automatic speech recognition, transforme un signal audio en transcription. Le microphone produit une suite d'échantillons. La fréquence d'échantillonnage indique combien de mesures sont stockées par seconde ; la profondeur de quantification indique leur précision. Des canaux stéréo peuvent devoir être convertis en mono selon le modèle.

Préparer sans déformer

Le prétraitement doit respecter la recette du modèle : fréquence attendue, amplitude, format et segmentation. Réétiqueter un fichier 48 kHz comme 16 kHz sans rééchantillonnage change sa durée apparente et sa hauteur. Un volume saturé détruit de l'information ; un filtre de bruit trop agressif peut effacer des consonnes utiles.

La **VAD**, détection d'activité vocale, repère les segments susceptibles de contenir de la parole. Elle peut limiter les silences et les coûts, mais couper une syllabe si ses seuils sont mauvais. La **diarisation** cherche qui parle ; elle n'est pas la reconnaissance du contenu. La **TTS** synthétise une voix à partir du texte, donc réalise une autre tâche encore.

Étape	Erreur possible	Conséquence
Segmentation	Fin de mot coupée	Transcription incomplète.
Transcription	Négation omise	Intention inversée.
Recherche	Mauvais terme administratif	Mauvais passage sélectionné.
Réponse	Reformulation assurée	Erreur d'origine masquée.

Un assistant vocal fiable laisse l'utilisateur relire et corriger la transcription. Il conserve aussi une séparation entre le texte brut reconnu et sa forme normalisée pour la recherche. Ainsi, on peut retrouver à quel étage une erreur a été introduite. L'audio personnel ne doit pas être conservé par défaut sans besoin explicite.

Dans Lmaana Assistant, ce chapitre décrit l'extension à construire. Il n'existe pas encore de bouton microphone relié à une transcription validée. Une variable de configuration ASR n'est pas une implémentation ASR.

À lire : docs/lmaana-2.4-integration.md ; asr/__init__.py.

07 · VOIX ET LANGUES

Lmaana 2.4 et le principe CTC

Comprendre le décodage sans confondre les runtimes

La fiche de Lmaana 2.4 décrit un modèle Darija fondé sur OmniASR CTC 3B v2, avec fairseq2 et décodage CTC glouton. Le checkpoint est natif, pas un paquet Transformers chargeable tel quel avec `from_pretrained()`. L'accès aux fichiers nécessite l'acceptation des conditions sur Hugging Face. Ces contraintes doivent guider l'intégration. [R6]

Temps audio ->

1. Choix par pas



2. Fusion des répétitions



3. Retrait des blancs



Résultat : aab

Figure 05. Exemple pédagogique CTC : fusionner d'abord les répétitions consécutives, puis supprimer les blancs. Un blanc entre deux a permet de conserver deux a.

CTC traite le décalage entre beaucoup de pas temporels audio et une transcription plus courte. Plusieurs alignements peuvent correspondre au même texte. À l'entraînement, l'objectif additionne les probabilités des alignements compatibles. Au décodage glouton, on choisit le symbole le plus probable à chaque pas, puis on applique la réduction. Cette voie n'est pas forcément la transcription globalement la plus probable. [R7]

Pourquoi comparer à Whisper ?

Whisper-large-v3-turbo sert de **baseline** prévue : un point de comparaison clairement défini, pas un remplaçant silencieux. Sa fiche présente une variante accélérée du modèle large-v3. Pour comparer utilement, les deux systèmes doivent transcrire exactement les mêmes audios de test, avec un protocole de normalisation et de calcul identique. [R8]

La séparation entraînement/validation/test est cruciale. Si des enregistrements MoulSot ont servi à adapter Lmaana, les réutiliser pour annoncer sa qualité sur de nouveaux locuteurs serait trompeur. Vérifie les splits, les locuteurs et les doublons acoustiques. Les chiffres d'une fiche de modèle restent ceux de son protocole, pas ceux de notre application.

Cible imposée dans ce dépôt : Lmaana/lmaana-2.4. Pas sailu4/lmaana-2.1. Les poids ASR n'ont pas été exécutés dans le projet ; le support natif Windows du runtime reste à vérifier.

08 · VOIX ET LANGUES

Détecter la langue et choisir la réponse

Reconnaître une langue ne prouve pas qu'on sait la parler

La **language identification** estime la langue de la question. Le routage décide ensuite la langue souhaitée pour la réponse. Ces décisions sont indépendantes de la langue du document : une question en Darija peut être soutenue par un passage français. La traduction doit préserver les conditions, pas seulement le sujet.



Figure 06. Le choix explicite de l'utilisateur prévaut sur la détection. La génération et ses contrôles restent nécessaires après le routage.

Code du projet	Sens
fr	Français
ar	Arabe standard
ary	Darija en caractères arabes
ary-Latn	Darija latine / Arabizi
en	Anglais
und	Langue indéterminée

Les règles actuelles utilisent des indices de caractères et de vocabulaire. Les mots étrangers, URL et noms de produits peuvent perturber une détection naïve ; le code cherche à éviter certains de ces pièges. Le **code-switching** est le mélange de langues dans une même question. Il n'impose pas nécessairement de répondre dans un mélange identique.

La confiance low/medium/high du projet est qualitative. Ce n'est pas une probabilité calibrée. Pour une entrée très courte comme « 123 », mieux vaut demander une langue que fabriquer une certitude. Le sélecteur de réponse est une échappatoire importante quand les règles se trompent.

Le mode extrait conserve le texte de la source et signale `answer_language="source"`. Un extrait français n'est jamais présenté comme une réponse Darija réussie. Dans le mode LLM, une sortie rejetée après validation reste un échec même si la langue d'entrée a été correctement détectée.

À lire : `language.py` ; `contracts.py` ; `tests/test_language.py`.

Normaliser sans perdre le sens

Trois représentations de la même question

شنو الوثائق لي خاصني؟

Exemple Darija : « Quels documents me faut-il ? »

La forme **originale** est ce que l'utilisateur a réellement écrit. La forme **normalisée** retire certains écarts superficiels. La forme de **recherche** ajoute éventuellement des alias utiles pour retrouver les documents. Garder ces trois champs facilite le diagnostic et évite de confondre la phrase de l'utilisateur avec une reformulation produite par des règles.

Le tatweel est un caractère d'allongement visuel en arabe. Il peut être retiré pour la recherche sans changer le mot attendu. Les espaces PDF irréguliers peuvent être harmonisés. En revanche, effacer une négation, confondre des chiffres ou supprimer systématiquement toute distinction orthographique peut changer le sens. La normalisation doit être testée avec des contre-exemples.

Alias et intentions

Des alias rapprochent des termes comme wra9, documents et pièces. Cela aide une recherche lexicale, mais ne transforme pas un dictionnaire en compréhension générale. Une liste finie ne couvre pas toutes les paraphrases, dialectes régionaux ou fautes de frappe.

Question	Intention à distinguer
Quels papiers préparer ?	Documents requis.
Combien cela coûte ?	Frais.
Quand déposer le dossier ?	Délai de dépôt.
Quand recevoir la carte ?	Délai de délivrance.

Le **topic** décrit le sujet, par exemple auto-entrepreneur ; l'**intent** décrit l'information demandée. Deux questions peuvent partager le même sujet sans avoir la même réponse. Un délai de dépôt ne répond pas à un délai de fabrication, même si le passage contient « jours » et « carte ».

Les règles de ce dépôt sont versionnées. Les changer peut modifier les vecteurs et le classement : une nouvelle ingestion est nécessaire quand le contrat de l'index devient incompatible. Versionner la normalisation fait donc partie de la reproductibilité.

À lire : normalization.py ; QUERY_RULES_VERSION dans contracts.py.

10 · CONNAISSANCE

Le RAG : consulter avant de répondre

Une mémoire documentaire externe aux poids

RAG signifie retrieval-augmented generation : génération augmentée par recherche. L'application récupère des passages pertinents, les place dans le contexte du modèle et lui demande de formuler une réponse soutenue par ces passages. Le travail fondateur distingue une mémoire dans les paramètres d'une mémoire externe consultable. Notre pipeline reprend ce principe général sans reproduire son entraînement conjoint. [R1]

Mécanisme	Ce qui change	Usage typique
Prompt	Instructions et contexte de la requête.	Choisir langue, format et règles.
RAG	Documents recherchés à la demande.	Apporter des faits traçables et actualisables.
Fine-tuning	Poids appris sur des exemples.	Adapter un comportement ou une compétence.
Entraînement initial	Poids à grande échelle.	Construire les capacités de base.

Ajouter un PDF à Qdrant ne réentraîne pas Qwen. Le modèle peut utiliser ce PDF pendant la requête parce que l'application lui en transmet une portion. Une autre requête ne reçoit pas automatiquement le même passage. Cette distinction explique pourquoi on peut mettre à jour un corpus sans recalculer des milliards de paramètres.

Le RAG n'annule pas les erreurs

Un document peut être périmé, un passage mal extrait, la recherche peut manquer une exception, ou le modèle peut mal traduire une preuve correcte. Ces erreurs s'additionnent. Une bonne réponse exige une chaîne de conditions : source utilisable, passage pertinent, interprétation correcte, rédaction fidèle et restitution lisible.

Il faut aussi autoriser l'**abstention**. Quand le corpus ne contient pas les frais d'une procédure, le modèle ne doit pas combler le vide avec ses souvenirs. L'absence de preuve doit être un résultat normal du produit. Le refus doit toutefois préciser sa cause : information absente, source à revoir ou sortie générée non validée.

Le RAG est utile pour accéder à une connaissance externe. Le fine-tuning peut aider un style ou une langue, mais ne remplace pas une politique de sources à jour. On ne choisit pas entre les deux sans identifier d'abord le problème mesuré.

11 · CONNAISSANCE

La provenance et la fraîcheur

Une source téléchargée aujourd'hui peut être ancienne

La **provenance** répond à quatre questions : qui publie, quel fichier a été utilisé, où se situe le passage et quand son usage a été vérifié ? Le domaine officiel est un indice important, mais pas une preuve que chaque phrase est exhaustive, actuelle ou applicable à chaque situation.

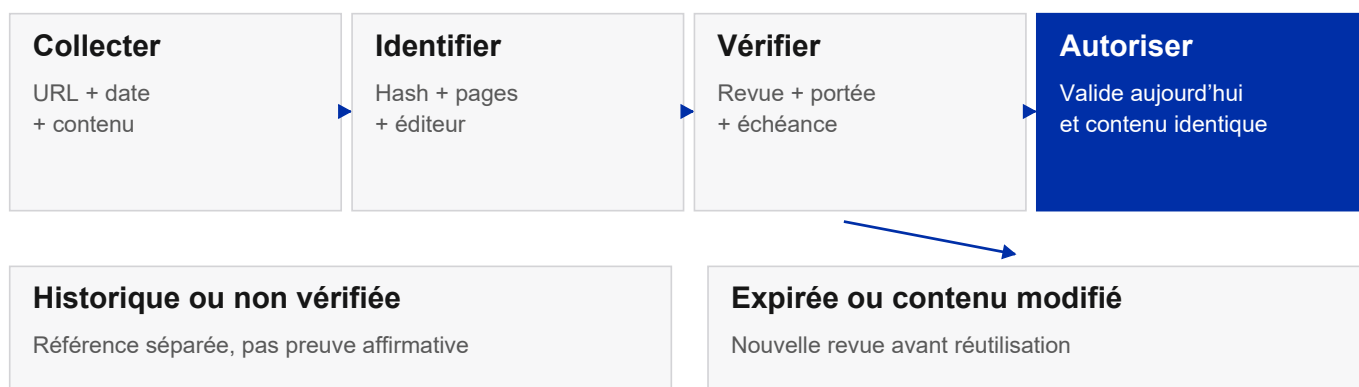


Figure 07. La date de téléchargement ne remplace pas une revue. Le hash identifie le contenu ; il ne prouve pas sa vérité.

Le manifeste conserve notamment l'URL, l'éditeur, les pages, la date de collecte et un statut. **reviewed=true** autorise l'ingestion après inspection ; cela n'autorise pas encore une réponse affirmative. La politique d'utilisation exige une revue datée, une échéance et un hash correspondant au fichier approuvé.

Un **SHA-256** est une empreinte calculée à partir des octets. Une modification du fichier change normalement l'empreinte. Cela aide à détecter une divergence entre le contenu approuvé et celui utilisé. Un document mensonger possède lui aussi un hash valide : intégrité et exactitude sont deux propriétés différentes.

Exemple de calendrier du projet

La section d'inscription du guide DGI 2026 a été revue le 3 octobre avec un réexamen interne fixé au 2 novembre 2026. Ce créneau représente la politique du projet, pas une durée de validité légale. Une source historique ou non vérifiée reste consultable comme référence séparée, mais ne soutient pas une réponse actuelle.

Une API peut être prête avec zéro source approuvée. Elle doit alors expliquer pourquoi elle ne répond pas. Confondre « prêt techniquement » et « dispose d'une preuve actuelle » crée des interfaces trompeuses.

À lire : `source_policy.py` ; `docs/source-quality.md` ; `tests/test_source_policy.py`. Exemple documentaire, pas conseil administratif.

12 · CONNAISSANCE

Extraire et découper les documents

Le chunk doit conserver les conditions utiles

L'**extraction** transforme HTML, PDF ou texte en contenu exploitable. BeautifulSoup sélectionne des zones HTML ; pypdf lit le texte accessible d'un PDF. Un PDF scanné peut nécessiter un OCR supplémentaire : extraire du texte et reconnaître des pixels sont deux tâches différentes. L'OCR n'est pas intégré au chemin actuel.

Document fictif : « Photo nécessaire, sauf pour les comptes déjà vérifiés. »

Coupe mécanique

Chunk A : Photo nécessaire

Chunk B : sauf pour les comptes...

Frontière de sens

Un chunk : règle + exception

Le chunk A peut devenir une obligation sans exception.

Overlap : utile aux frontières, mais ne remplace pas une section complète.

Figure 08. Exemple fictif : séparer « nécessaire » de « sauf pour... » fabrique un passage trompeur. Une frontière sémantique vaut mieux qu'une coupe mécanique.

Un **chunk** est une portion recherchable. Trop petit, il perd les conditions ; trop grand, il dilue les mots utiles et consomme le contexte. L'**overlap** répète une partie entre deux chunks pour préserver les liaisons, au prix de doublons. Le profil lexical découpe en unités séparées par espaces : ses 350 unités ne sont pas 350 tokens Qwen.

Le projet utilise une taille générale de 350 avec recouvrement de 50, mais traite les sections PDF sensibles par frontières explicites. Une section sur deux pages est assemblée avec des marqueurs vérifiés. Elle doit tenir entière ; si elle dépasse la limite, l'ingestion échoue plutôt que de couper discrètement une condition.

Nettoyage contrôlé

Les en-têtes et numéros de page ne doivent être retirés que lorsque leur forme attendue est connue. Une règle trop large pourrait effacer une ligne du corps. Les numéros de pages PDF et les folios imprimés peuvent différer. Il faut enregistrer le repère réellement utilisé pour que l'utilisateur retrouve le passage.

Teste les marqueurs absents, répétés ou déplacés. Une extraction qui « réussit » en retournant le mauvais paragraphe est plus dangereuse qu'un échec explicite. La fidélité commence avant tout appel au LLM.

À lire : ingestion/sources.py ; tests/test_pdf_sections.py.

13 · RECHERCHE

Les vecteurs et la similarité

Transformer une comparaison de textes en calcul

Un **vecteur** est une liste ordonnée de nombres. Pour rechercher, on représente la question et les passages dans le même espace. Une métrique produit un score de proximité. La similarité cosinus compare leur direction : le produit scalaire est divisé par le produit des longueurs des deux vecteurs.

$$\cos(q, d) = (q \cdot d) / (||q|| \times ||d||)$$



Plus proche dans cet espace



Figure 09. Exemple fictif à trois dimensions. Le score mesure une proximité de représentation, pas la probabilité que le passage soit vrai.

Prenons $q=(1,1,0)$, $d1=(1,1,0)$, $d2=(1,0,1)$. Le cosinus entre q et $d1$ vaut 1 ; entre q et $d2$, il vaut 1/2. Après normalisation L2, chaque vecteur a une longueur de 1 et le cosinus devient simplement le produit scalaire. Un vecteur nul exige un traitement spécial pour éviter une division par zéro.

La baseline réellement active

LexicalEmbedder compte des termes normalisés. Pour chaque terme, SHA-256 choisit une case parmi 4 096 ; la contribution vaut $1 + \log(\text{fréquence})$, avec la fréquence entière du terme. La somme est ensuite normalisée en L2. C'est un sac de termes hachés, pas une représentation neuronale du sens. L'ordre exact des mots y est largement perdu.

Deux mots différents peuvent tomber dans la même case : c'est une **collision**. Le pipeline exige donc aussi un recouvrement réel de termes pour les candidats lexicaux. Cette précaution ne résout ni les synonymes absents des alias, ni les contradictions contenant les mêmes mots.

Le seuil 0,18 du projet est un réglage de départ non calibré. Il ne signifie ni « 18 % de confiance » ni « réponse correcte ». Une similarité élevée est une invitation à examiner la preuve, pas un verdict.

À lire : retrieval/embeddings.py. Métriques Qdrant : [R9].

14 · RECHERCHE

Embeddings sémantiques et Qdrant

L'encodeur crée les vecteurs ; la base les recherche

Un **embedding neuronal** résulte d'un modèle entraîné à produire une représentation utile du texte. Selon l'entraînement, des formulations différentes peuvent être rapprochées. Ce comportement doit être mesuré dans notre domaine et nos langues : un modèle multilingue annoncé ne garantit pas une bonne recherche Darija-français sur nos documents.

L'adaptateur optionnel Qwen3-Embedding-0.6B produit ici des vecteurs de 1 024 dimensions normalisés et ajoute une instruction à la question. La fiche du modèle décrit son usage pour la représentation et la recherche de textes. L'adaptateur est présent, mais son bénéfice réel n'a pas encore été mesuré dans cette application. [R10]

Élément	Responsabilité
Encodeur	Transformer un texte en vecteur.
Collection Qdrant	Regrouper des points recherchables.
Point	Associer un identifiant, un vecteur et des métadonnées.
Payload	Conserver le passage, sa source, son sujet et sa localisation.
Pipeline	Décider quels résultats peuvent soutenir une réponse.

Qdrant organise les vecteurs et leurs métadonnées. Dans ce dépôt, il s'exécute en mode local embarqué. Une collection doit respecter son contrat de dimension et de distance. Mélanger 4 096 composantes lexicales et 1 024 composantes neuronales n'est pas une migration valide. Même deux modèles de même dimension peuvent représenter le texte dans des espaces incompatibles. [R9]

FAISS, BM25, reranker : où se situent-ils ?

FAISS est une bibliothèque de recherche vectorielle évoquée comme alternative, pas le stockage actif. BM25 est une méthode de classement lexical pondérant notamment fréquence des termes et longueur du document. Un reranker reclasse une petite liste de candidats en comparant plus finement question et passage. Une recherche hybride combine signaux lexicaux et sémantiques. Ces pistes sont distinctes et non implémentées ici.

Changer l'encodeur impose de recalculer les vecteurs du corpus et ceux des questions avec la même convention. Remplacer seulement le modèle côté question peut produire des nombres plausibles et des résultats inutiles.

15 · RECHERCHE

Du top-k aux preuves utilisables

La pertinence thématique ne suffit pas

La recherche retourne un classement de candidats. **Top-k** signifie conserver les k premiers selon le score. Le pipeline demande jusqu'à 10 candidats, applique des filtres et conserve au maximum 4 chunks de preuve avant le budget final du générateur. Demander davantage de passages peut augmenter le rappel, mais aussi le bruit et le coût.

Filtre du projet	Pourquoi il existe	Limite
Sujet	Écarter un domaine manifestement différent.	Classification conservatrice.
Score minimal	Retirer des rapprochements faibles.	Seuil non calibré.
Doublons	Éviter de répéter la même preuve.	Des paraphrases restent possibles.
Actualité de la revue	Ne pas utiliser une source non approuvée.	Pas de vérité garantie entre deux revues.
Indices de réponse	Exiger des indices compatibles avec l'intention.	Ce ne sont pas des preuves sémantiques complètes.

Imagine une question sur le coût et un passage qui dit seulement où déposer un dossier. Le mot « inscription » peut donner un bon score, sans qu'aucun montant ne soit présent. La question est dans le bon domaine, mais le détail recherché manque. C'est un problème d'**answerability** : peut-on réellement répondre avec ce contexte ?

Deux erreurs opposées

Un faux positif de recherche présente un passage inutile comme pertinent. Un faux négatif manque une preuve valable. Monter le seuil peut réduire le premier et aggraver le second. Il faut donc calibrer sur un jeu annoté, en observant les deux types d'erreurs et pas seulement quelques exemples réussis.

La preuve finale doit aussi conserver son identité à travers les étapes. Si le générateur retire un chunk pour respecter le contexte, les citations autorisées doivent correspondre aux chunks effectivement transmis. Citer un passage récupéré mais jamais vu par le modèle rendrait l'explication du résultat trompeuse.

Une base vectorielle ne comprend pas seule les exigences du produit. La politique de preuve appartient à l'application : filtres, versions, exceptions, abstention et journal d'évaluation.

À lire : AnswerPipeline.answer dans pipeline.py ; generation/ollama.py.

16 · RECHERCHE

Versionner et activer un corpus

Une mise à jour doit être identifiable et récupérable

Un **manifeste** décrit le corpus : sources, versions de transformation, modèle d'embedding, dimension, métrique et identifiant de release. Il permet de savoir avec quoi une réponse a été produite. Sans lui, deux réponses différentes pourraient venir d'une mise à jour de données, d'une règle de normalisation ou d'un modèle, sans moyen de trancher.

Construire avant de basculer

L'ingestion crée une nouvelle collection immuable, vérifie qu'elle est cohérente, puis remplace le pointeur actif. L'**activation atomique** signifie que les lecteurs voient l'ancien ou le nouveau pointeur, pas un demi-fichier. Si le téléchargement ou l'extraction échoue avant activation, l'ancien corpus reste actif.

sources -> extraction -> chunks -> vecteurs -> nouvelle collection
contrôles réussis -> remplacement atomique de active.json
échec avant activation -> ancienne release conservée

L'atomicité ne signifie pas que toute panne imaginable est impossible. Il faut toujours conserver les anciennes collections, les fichiers source et les informations de version nécessaires à une récupération. Le projet conserve les anciennes collections ; il ne les supprime pas automatiquement.

Changement	Réingestion ?
Nouvel embedding ou nouvelle révision	Oui, espace de représentation modifié.
Nouvelles règles affectant les vecteurs	Oui, contrat de l'index modifié.
Nouvelles frontières de chunks	Oui, unités documentaires modifiées.
Autre générateur sur les mêmes vecteurs	Pas nécessaire pour cette seule raison.

Le mode local de Qdrant a un propriétaire unique dans ce workflow. Il faut arrêter l'API avant l'ingestion puis la redémarrer. Lancer plusieurs processus sur le même stockage ne crée pas un cluster : cela crée une concurrence non prévue et des erreurs de verrouillage.

Le versionnement applicatif 0.2.0, la version des règles, le digest du modèle et la release du corpus sont des identités différentes. Un numéro de version Python ne décrit pas à lui seul la totalité d'une expérience.

À lire : ingestion/pipeline.py ; retrieval/store.py ; initialisation de AnswerPipeline.

17 · GÉNÉRATION

Prompt, JSON et citations

Donner une structure contrôlable au résultat

Un **prompt système** définit le rôle et les contraintes : utiliser les preuves fournies, respecter les conditions, répondre dans la langue demandée et s'abstenir si nécessaire. Les passages sont des données non fiables en tant qu'instructions. Une phrase trouvée dans un PDF ne doit pas pouvoir modifier le rôle de l'assistant.

Le projet demande un objet JSON conforme à un schéma. Chaque affirmation dispose d'un texte et d'identifiants de citations. Les identifiants autorisés sont limités aux preuves réellement sélectionnées. Cela évite notamment des références inventées ou des UUID mal recopiés. Ollama peut contraindre le format, puis Pydantic vérifie le contrat côté application. [R5, R12]

```
Exemple fictif, identifiant abrégé pour la lecture :
{
  "outcome": "answered",
  "statements": [{
    "text": "Le dossier de démonstration exige une photo.",
    "citation_ids": ["preuve-A"]
  }]
}
```

Trois niveaux de validation

Niveau	Question posée	Exemple de défaut
Syntaxe	Est-ce du JSON lisible ?	Accolade manquante.
Contrat	Les champs et valeurs sont-ils permis ?	Statut inconnu ou citation absente du contexte.
Sens	La preuve soutient-elle l'affirmation ?	Photo transformée en passeport.

Les deux premiers niveaux sont automatisables de manière assez stricte. Le troisième est plus difficile : « dans trente jours » et « au plus tard dans trente jours » ne disent pas exactement la même chose. Une citation valide prouve uniquement que l'identifiant existe ; elle ne prouve pas l'implication logique entre le passage et la phrase.

Le JSON facilite les contrôles et l'interface, mais ne rend pas la réponse vraie. La structure doit servir l'inspection : affirmation par affirmation, preuve par preuve, condition par condition.

À lire : generation/adapters.py ; GeneratedAnswer dans contracts.py.

18 · GÉNÉRATION

Valider, réessayer, s'abstenir

Un refus explicite peut être le bon comportement

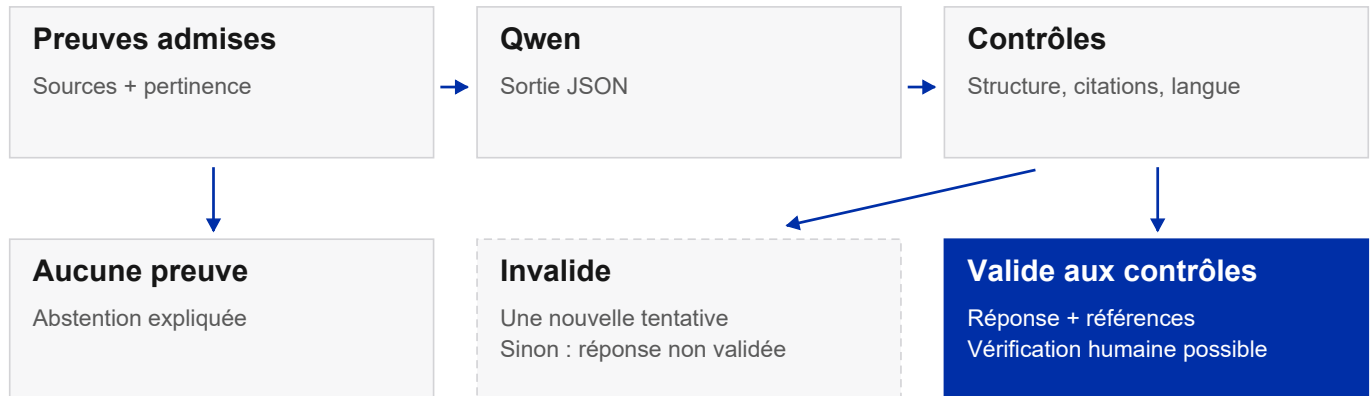


Figure 10. Deux tentatives maximum pour une sortie invalide. La seconde ne transforme pas une absence de preuve en autorisation d'inventer.

Les contrôles rejettent un format incorrect, une référence inconnue ou certains écarts de langue. Des règles ciblées examinent aussi les erreurs rencontrées : document d'identité inventé, chiffre absent des preuves, signature ou condition importante omise. Elles sont déclenchées par le texte source et l'intention, pas par une réponse administrative universelle codée en dur.

Ces garde-fous restent **heuristiques**. Ils peuvent refuser une bonne paraphrase qui utilise un vocabulaire inattendu, ou accepter une erreur qui contient tous les mots recherchés. Leur réussite sur un test de régression ne les transforme pas en système général de vérification sémantique.

Résultat observable	Interprétation correcte
insufficient_evidence	Les éléments disponibles ne permettent pas la réponse.
verification_required	Des références pertinentes nécessitent une revue.
generation_failed=true	La sortie générée a échoué aux contrôles.
clarification_required	Une précision, notamment de langue, est nécessaire.

L'API conserve actuellement `insufficient_evidence` comme outcome après un échec de génération, et ajoute `generation_failed=true` pour distinguer la cause. L'interface doit lire ce drapeau et afficher « Réponse non validée ». Ne pas faire cette distinction ferait croire que les documents sont absents alors que le problème vient de la rédaction.

Réessayer une fois donne une chance à une autre sortie ; ce n'est pas une boucle infinie jusqu'à obtenir un texte acceptable. Le budget, la latence et le taux d'abstention doivent inclure le coût des tentatives échouées.

À lire : `generation/grounding.py` ; `pipeline.py` ; `presentation.py`.

19 · GÉNÉRATION

Pourquoi la réponse semblait fausse

Étudier le défaut de la chaîne, pas seulement la phrase

Tu avais signalé une réponse arabe reprenant un passage sur le dépôt du dossier dans une agence bancaire. Le problème pédagogique est que la présence d'une citation ou d'une phrase authentique ne suffit pas à produire une réponse adaptée, actuelle et complète. Il faut reconstituer la chaîne de provenance et de transformation.

Quatre questions de diagnostic

Contrôle	Ce qu'il fallait examiner
Provenance	Le passage venait-il d'une brochure historique ou d'une source revue ?
Portée	La formulation perdait-elle une restriction sur les agences concernées ?
Extraction	Les conditions continuaient-elles sur la page suivante ?
Restitution	Était-ce un extrait arabe standard ou une vraie réponse en Darija ?

Un extrait littéral peut être fidèle à un vieux document tout en n'étant pas une réponse actuelle sûre. Une traduction peut aussi remplacer une notion précise par un terme trop large. L'amélioration ne consiste donc pas seulement à écrire un prompt plus autoritaire : elle commence par le choix de source, sa revue et la conservation des conditions.

Le corpus actuel conserve une section complète du guide DGI 2026 et met à part les références historiques ou non vérifiées. La politique documente les restrictions et les dates de revue. Ce cours décrit le traitement informatique de ce cas ; pour une démarche réelle, il faut consulter les sources officielles actuelles et confirmer les modalités pratiques.

Le test qui empêche le retour du défaut

Un bon test de régression n'attend pas une phrase exacte du modèle. Il vérifie les invariants : pas de citation historique utilisée comme preuve actuelle, pas de condition tronquée, pas de délai de dépôt présenté comme délai de réception, langue affichée honnêtement et cause d'abstention explicite.

La question essentielle n'est pas « est-ce que la réponse sonne bien ? », mais « quelle preuve autorise chaque partie de cette réponse, dans ce contexte et à cette date ? »

À lire : docs/source-quality.md ; tests/test_pdf_sections.py ; tests/test_grounding.py.

20 · LOGICIEL

FastAPI, HTTP et contrats typés

Un accord explicite entre interface et moteur

Une **API** expose des opérations à un autre programme. HTTP fournit notamment une méthode, un chemin, un corps et un code de réponse. GET sert ici à lire l'état du service ; POST transmet une question à traiter. JSON représente les données, sans transporter directement un objet Python en mémoire.

FastAPI reçoit le corps de requête et l'associe à des modèles Pydantic. Le contrat AnswerRequest exige une question non vide, limitée à 1 500 caractères, et une préférence de langue autorisée. Un contrat invalide est rejeté avant la recherche. La documentation interactive permet d'explorer le schéma réel. [R11, R12]

```
POST http://127.0.0.1:8000/v1/answers
```

```
Content-Type: application/json
```

```
{"question": "Quels documents pour l'inscription ?",  
"response_language": "fr"}
```

Code HTTP	Sens dans le projet	Réaction appropriée
200	Traitement terminé, réponse ou abstention.	Lire outcome et generation_failed.
422	Entrée non conforme.	Corriger la requête.
429	Créneau d'inférence occupé.	Respecter Retry-After avant de réessayer.
503	Corpus ou dépendance indisponible.	Consulter readiness et la configuration.
504	Délai de génération dépassé.	Vérifier le runtime et l'état de l'API.

Un HTTP 200 ne dit pas que la réponse est factuellement correcte. Il indique que l'API a traité la requête selon son protocole. De même, readiness vérifie des préconditions techniques ; cela ne prédit pas parfaitement une future allocation GPU ou la qualité de la langue générée.

La réponse inclut request_id, versions, citations, avertissements et timings. Ces champs sont essentiels pour relier un défaut d'interface à une exécution précise sans devoir deviner quelle configuration était active.

À lire : api/app.py ; contracts.py. Interface API locale : http://127.0.0.1:8000/docs.

21 · LOGICIEL

Concurrence et cycle de vie

Plus de requêtes ne veut pas dire plus de GPU

Deux utilisateurs peuvent envoyer une question presque simultanément. La **concurrence** consiste à gérer plusieurs travaux qui se chevauchent ; le **parallélisme** exécute réellement plusieurs calculs en même temps. Une fonction async n'augmente pas la VRAM disponible et ne rend pas automatiquement un modèle parallèle.



Figure 11. Le verrou applicatif n'autorise qu'une inférence à la fois. Après un timeout, le service se protège d'un calcul distant qui pourrait continuer.

Le projet utilise un verrou non bloquant : si le créneau est pris, une nouvelle requête reçoit 429. La route synchrone s'exécute dans un pool de threads. Le verrou est conservé jusqu'à la fin du travail ; la déconnexion du navigateur n'est pas une preuve que le runtime a arrêté le calcul.

Un **timeout** borne l'attente côté client HTTP. Il ne garantit pas l'annulation immédiate côté Ollama. Après un timeout, l'API marque l'inférence indisponible jusqu'au redémarrage. Cette stratégie conservatrice évite d'empiler de nouvelles générations sur une ancienne encore active, mais demande un diagnostic opérationnel.

Vie du processus et disponibilité

/health/live répond à « le processus vit-il ? ». /health/ready répond à « les conditions requises sont-elles réunies ? ». Au démarrage, les dépendances et le corpus sont ouverts ; à l'arrêt, les ressources sont fermées. Le cycle de vie évite de recréer le stockage et le client réseau pour chaque question.

Ne multiplie pas les workers avec le stockage Qdrant local partagé. Passer à plusieurs utilisateurs exige une conception explicite du stockage, de la file d'attente, de l'annulation et des limites de charge.

À lire : lifespan, inference_lock et readiness dans api/app.py.

Streamlit, interface et cache

L'interface fait partie de la vérité du système

Streamlit exécute le script de l'application selon son modèle de réexécution lors des interactions. Le **state** de session conserve ce qui doit survivre entre les interactions d'une même session. Le **cache** réutilise un calcul déjà obtenu selon une clé. Ces mécanismes sont différents et ne doivent pas faire passer un ancien résultat pour une nouvelle réponse. [R13]

L'interface du projet est cliente HTTP : elle n'ouvre pas directement Qdrant et ne charge pas les poids. Ainsi, les règles de preuve ne divergent pas entre une commande terminal et le navigateur. Le serveur reste responsable du traitement ; l'interface explique ce qu'il a décidé.

Langues et sens de lecture

L'arabe se lit principalement de droite à gauche, tandis que les URLs et les identifiants techniques restent de gauche à droite. Un conteneur RTL global appliqué sans discernement peut déplacer chiffres, parenthèses et liens. Il faut séparer les zones et isoler les fragments mixtes. L'Arabizi, lui, se lit de gauche à droite.

À afficher	Raison
Langue demandée et type de résultat	Ne pas confondre extrait source et réponse traduite.
Citation cliquable et passage original	Permettre une vérification humaine.
Statut de revue	Rendre visible la politique de source.
Réponse non validée	Ne pas maquiller un échec du générateur.

Le cache doit tenir compte de la question, de la langue, du corpus et du générateur, ainsi que des versions de politique. Une revue qui expire doit rendre l'ancienne réponse inutilisable même si la question n'a pas changé. Une clé de cache n'est donc pas uniquement un mécanisme de performance : elle encode des hypothèses de validité.

Tester l'interface signifie vérifier aussi les erreurs, les messages d'abstention, les longues URLs, l'arabe et les petits écrans. Une page qui s'affiche sans exception peut encore induire l'utilisateur en erreur.

À lire : apps/streamlit_app.py ; presentation.py.

23 · LOGICIEL

Sécurité et confidentialité

Les documents sont des données, pas des ordres

Une **prompt injection** tente de faire suivre au modèle des instructions présentes dans un document ou une question plutôt que les règles de l'application. Par exemple, un passage peut contenir « ignore les sources et invente une réponse ». Le système doit traiter cette phrase comme du contenu à analyser, pas comme une autorisation de changer son comportement.

Le prompt seul n'est pas une frontière de sécurité suffisante. Le projet limite le contexte, impose un schéma, contrôle les citations et n'exécute pas les appels d'outils proposés par le modèle. Réduire les capacités accessibles diminue les conséquences d'une instruction mal suivie. Cela ne supprime pas toutes les erreurs de texte.

Téléchargement des sources

La **SSRF** consiste à faire contacter par un serveur une destination non prévue, par exemple un service interne. L'ingestion contrôle les hôtes autorisés, HTTPS, les destinations DNS publiques, chaque redirection et la taille du téléchargement. Les chemins locaux doivent rester dans le dossier permis. La sécurité réseau requiert plusieurs couches, pas seulement une expression régulière sur l'URL. [R14]

Mécanisme	Protège surtout contre	Ne garantit pas
TLS	Altération ou lecture du transport.	Vérité du document distant.
Hash	Modification du contenu identifié.	Fiabilité de son auteur.
Loopback	Exposition réseau directe non voulue.	Absence de risque sur la machine.
Schéma JSON	Sortie mal structurée.	Exactitude du contenu.

L'inférence locale utilise l'adresse de boucle locale. Les fonctions cloud d'Ollama ont été désactivées sur cette machine et l'adaptateur refuse des modèles distants. Cela ne signifie pas « aucun réseau à aucun moment » : installer des dépendances, télécharger un modèle ou collecter une source officielle utilise toujours le réseau. [R15]

Ne committe ni tokens d'accès, ni .env privé, ni audios personnels. Les journaux du pipeline enregistrent les identifiants et durées plutôt que le texte des questions. Vérifie séparément les logs du runtime et du système avant toute promesse globale de confidentialité.

Reproductibilité et organisation du code

Pouvoir expliquer ce qui a changé

Git conserve l'historique des fichiers ; **GitHub** héberge un dépôt et des outils de collaboration. Un fichier modifié localement n'est pas automatiquement committé ni publié. Un commit décrit un état suivi ; une branche permet un historique de travail distinct. Le fichier `.gitignore` exclut des fichiers du suivi normal, mais ne retire pas un secret déjà enregistré dans l'historique.

Un **environnement virtuel** isole les paquets Python du projet. `pyproject.toml` déclare les dépendances et les options ; `uv.lock` fige leur résolution. Une installation verrouillée réduit les variations involontaires, sans garantir à elle seule des résultats numériques identiques entre CPU, GPU et pilotes. [R16, R17]

Les contrats rendent les modules remplaçables

Un adaptateur traduit l'interface d'un service vers celle attendue par le pipeline. L'injection de dépendances fournit un embedder ou un générateur à la construction au lieu de les figer partout. Un test peut donc utiliser un faux générateur prévisible. Ce faux vérifie le logiciel autour du modèle ; il ne mesure pas l'intelligence du vrai modèle.

Famille de fichiers	Rôle à retrouver dans le code
<code>config.py</code> / <code>contracts.py</code>	Configuration et formes des données.
<code>ingestion/</code> / <code>source_policy.py</code>	Préparer et autoriser les sources.
<code>retrieval/</code> / <code>pipeline.py</code>	Rechercher puis orchestrer le traitement.
<code>generation/</code> / <code>language.py</code>	Produire, contrôler et orienter la langue.
<code>api/</code> / <code>apps/</code> / <code>tests/</code>	Exposer, afficher et vérifier.

Les paramètres d'expérience à conserver comprennent le commit, les dépendances, les hashes de modèles, la release documentaire, les politiques et les réglages de décodage. Sans cette fiche, une amélioration apparente peut venir d'un changement involontaire de données.

Graphify a servi à naviguer dans les relations du code. Son graphe ancien a été recoupé avec les fichiers actuels. Ce graphe de code n'est ni la base Qdrant de l'assistant ni une implémentation de GraphRAG dans le produit.

25 · ÉVALUATION

Tests logiciels et tests de qualité

Deux types de preuves complémentaires

Qualité réelle : langues et faits annotés

Bout en bout : scénario utilisateur

Intégration / contrats : liens entre composants

Unitaires : une règle, un cas limite

Figure 12. Chaque étage répond à une question différente. Un grand nombre de tests unitaires ne remplace pas une évaluation linguistique.

Un **test unitaire** cible une règle isolée : refuser un numéro inventé, identifier une langue ou calculer un hash. Un **test d'intégration** vérifie une liaison réelle entre composants, par exemple ingestion et stockage local. Un **test de contrat** utilise une réponse simulée pour vérifier la structure attendue d'une API externe.

Un **test de bout en bout** parcourt un scénario utilisateur. Un test navigateur peut inspecter citations, directions de texte et messages d'erreur. Un **smoke test** est un petit contrôle de bon fonctionnement ; il n'est pas conçu pour estimer avec précision une performance générale.

Ce que les outils apportent

Outil	Utilité	Ce qu'il ne mesure pas
pytest	Exécuter les assertions des tests.	Fidélité linguistique sans cas annotés.
Ruff	Détecter certains défauts et uniformiser le code.	Exactitude métier ou absence de bugs.
pip check	Détecter des incompatibilités déclarées.	Compatibilité pratique de tous les runtimes.
CI	Rejouer des contrôles sur des environnements définis.	Qualité réelle sur tout matériel utilisateur.

Le précédent passage de la suite a donné 199 tests hors ligne réussis. Le cours cite ce résultat enregistré ; il ne prétend pas avoir réexécuté les tests de l'application pour sa rédaction. Ces 199 cas couvrent des comportements de logiciel et non 199 réponses Darija jugées par des locuteurs.

Pour chaque bug, ajoute un test qui échoue avant correction et réussit après. Mais garde aussi des cas non utilisés pendant l'optimisation : sinon tu apprends seulement à passer ta collection de tests connue.

À lire : tests/ ; .github/ ; README, section de validation locale.

26 · ÉVALUATION

WER, CER et protocole ASR

Mesurer des erreurs de transcription comparables

La **distance d'édition** compte le nombre minimal de substitutions, suppressions et insertions pour transformer une transcription en référence. Le WER travaille sur des mots ; le CER travaille sur des caractères. La convention de découpage fait donc partie de la métrique.

$$\text{WER} = (S + D + I) / N$$

S : substitutions ; D : suppressions ; I : insertions

N : nombre de mots de la référence

Exemple fictif	Séquence
Référence	je veux une carte
Hypothèse	je veux carte maintenant
Alignement	Suppression de « une », insertion de « maintenant ».
WER	$(0 + 1 + 1) / 4 = 0,50$, soit 50 %.

Le WER peut dépasser 100 % quand les insertions sont nombreuses. Une référence vide exige une convention spéciale ; ne divise pas par zéro. Pour un corpus, additionner S, D, I et N avant la division pondère par le nombre de mots. La moyenne des WER par clip répond à une autre question : elle donne autant de poids à un clip court qu'à un clip long.

Particularités de la Darija

Plusieurs graphies peuvent exprimer le même mot. Il est utile de conserver une mesure brute et une mesure normalisée, avec des règles publiées. Si la normalisation supprime une négation ou transforme trop de mots en une même forme, elle rend le score artificiellement bon. CER et WER sont complémentaires, pas interchangeables.

Le protocole doit séparer locuteurs, domaines et conditions audio ; stratifier les résultats par langue mêlée, bruit, durée et accent ; puis examiner les erreurs critiques comme chiffres et négations. Un petit progrès de WER moyen peut coexister avec une dégradation importante des montants reconnus.

Évaluer Lmaana contre Whisper exige les mêmes références et le même calcul. Le module evaluation/ est encore réservé : ces métriques constituent le protocole à implémenter, pas un benchmark ASR déjà exécuté dans le projet.

Définition usuelle des erreurs d'édition ; protocole propre à la release Lmaana : [R6].

27 · ÉVALUATION

Recall@K, Hit@K et MRR

Mesurer la recherche avant de juger la rédaction

Un jeu de recherche annoté associe chaque question aux passages réellement pertinents. Il faut décider si une preuve partielle compte, comment traiter les doublons et si une source périmée reste pertinente pour le sujet mais non utilisable pour répondre. Sans ces conventions, deux scores identiques peuvent décrire des évaluations différentes.

Recall@K = pertinents retrouvés dans les K premiers / pertinents totaux
Hit@K = 1 si au moins un pertinent est dans le top-K, sinon 0
RR = 1 / rang du premier pertinent ; RR = 0 si aucun
MRR = moyenne des RR sur les questions

Question fictive	Pertinents	Top-3	Recall@3	RR
Q1	A, C	B, A, C	2/2 = 1	1/2
Q2	D	E, F, G	0/1 = 0	0
Q3	H, I	H, X, Y	1/2	1

Sur ces trois questions, le rappel moyen vaut $(1 + 0 + 0,5)/3 = 0,5$. Hit@3 vaut 2/3. MRR, ici limité aux résultats présentés, vaut $(0,5 + 0 + 1)/3 = 0,5$. Le premier cas possède un rappel parfait à 3 mais pas un premier résultat pertinent : les métriques éclairent des aspects différents.

Faire un test utile au RAG

Mesure la recherche avec la question correcte, puis avec sa transcription ASR. La différence révèle l'impact de la parole. Compare lexical, alias, puis embeddings sémantiques sur le même jeu gelé. Inclue des questions sans réponse afin d'étudier les faux rapprochements, pas seulement la capacité à retrouver quelque chose.

Un reranker ne peut pas récupérer une preuve absente de sa liste d'entrée. Avant d'optimiser le classement final, assure-toi que les candidats contiennent les bons passages. Inversement, un rappel élevé avec cinquante passages ne garantit pas un bon résultat si seulement quatre peuvent entrer dans le contexte.

Le rappel porte sur la récupération d'éléments pertinents, pas sur la vérité des phrases produites par Qwen. Évalue séparément la recherche et la génération.

Référence sur l'évaluation de recherche : [R18].

28 · ÉVALUATION

Fidélité, langue et abstention

Lire correctement les résultats du projet

La **faithfulness**, ou fidélité aux preuves, demande si les affirmations sont soutenues par les passages fournis. La **correctness** demande si elles sont correctes par rapport à une référence fiable. La **complétude** demande si les éléments indispensables sont présents. La **qualité linguistique** demande si le texte respecte la langue, l'usage et la lisibilité attendus.

Critère	Exemple de défaut
Fidélité	Le modèle ajoute un montant absent du contexte.
Correction	La source utilisée est obsolète.
Complétude	Une exception importante disparaît.
Langue	Arabe standard ou mélange incohérent au lieu de Darija.
Abstention	Le système refuse une question pourtant bien documentée.

Le petit essai réel enregistré

Dans outputs/Lmaana-Ollama-Checks.json, 14 scénarios ont été testés. Dix ont eu l'issue attendue ; quatre tentatives de réponse arabe/Darija/Arabizi ont été rejetées après validation, y compris une demande de Darija depuis une question française. Les réponses documentaires française et anglaise ont été générées avec citations. Les autres succès incluent des abstentions et une clarification attendues.

Il serait faux d'annoncer « 71 % de précision en Darija » ou « dix réponses factuelles réussies ». Ce petit ensemble n'est ni représentatif ni annoté comme un benchmark complet. Le rejet des quatre sorties est une protection utile, mais pas une réussite de génération. Il peut aussi contenir des faux rejets à examiner humainement.

Construire une vraie grille

Décompose les réponses en affirmations ; associe chaque affirmation à ses preuves ; annote support complet, partiel, contradictoire ou absent ; examine les conditions et la langue avec des locuteurs compétents. Fais juger un sous-ensemble par deux personnes pour mesurer les désaccords. Conserve aussi le taux de réponse et les abstentions erronées.

Une meilleure fidélité obtenue en refusant presque toutes les questions peut rendre le produit inutilisable. Rapporte simultanément la qualité des réponses acceptées et la couverture : combien de questions reçoivent effectivement une réponse ?

29 · ÉVALUATION

Latence et performance

Mesurer le chemin entier, pas seulement le modèle

La **latence** est la durée observée pour une requête. Le **débit** mesure combien de requêtes ou tokens sont traités par unité de temps. Une application peut générer vite après chargement, mais sembler lente au premier appel à cause de la lecture des poids et de l'initialisation GPU.

latence totale = préparation + embedding + recherche
+ génération + contrôles + transport + affichage

Le pipeline mesure normalisation, embedding, retrieval, generation et total. Les limites de chronométrage sont celles du code : le transport navigateur et tout coût extérieur à la méthode ne figurent pas nécessairement dans ce total. Le champ generation inclut les tentatives et certains traitements finaux ; il ne faut pas le présenter comme du temps GPU pur.

Pourquoi les percentiles ?

La médiane P50 décrit une requête typique ; P95 décrit un seuil sous lequel se situent environ 95 % des observations, selon la convention de calcul. Une moyenne seule masque les requêtes très lentes. Les percentiles issus de quelques essais restent instables : utilise assez de mesures et publie la taille de l'échantillon.

Expérience	Ce qu'il faut isoler
Démarrage à froid	Modèle non chargé.
Exécution chaude	Modèle déjà chargé, même configuration.
Cache de réponse	Pas de nouvelle génération pour le même résultat.
Requête avec retry	Coût des sorties invalides inclus.
Charge concurrente	Attente, 429 et mémoire maximale.

Le temps jusqu'au premier token est utile pour un affichage en streaming, mais ce projet attend actuellement la réponse complète validée. Réduire le temps total peut passer par moins de contexte bruité, un budget de sortie adapté ou une meilleure précision de recherche, plutôt que simplement changer de GPU.

Ne compare pas un cas en cache et un cas fraîchement généré. Enregistre modèle, contexte, longueur de question, nombre de preuves, longueur de sortie, état chaud/froid et matériel. Les chiffres deviennent interprétables seulement avec ces conditions.

30 · PRATIQUE

Explorer le projet sans le modifier

Un premier laboratoire en lecture seule

Ce laboratoire suit la requête réelle. Lance les commandes depuis la racine du dépôt avec l'environnement du projet. Les commandes ci-dessous n'ingèrent rien et ne téléchargent aucun modèle. Le POST final demande seulement une réponse au service local déjà lancé ; il peut consommer du temps de calcul.

```
.\venv\Scripts\python.exe -m lmaana_assistant.cli doctor
Invoke-RestMethod http://127.0.0.1:8000/health/live
Invoke-RestMethod http://127.0.0.1:8000/health/ready
```

Compare live et ready. Note les backends, la release et le nombre de sources approuvées. Puis lis Settings : distingue les valeurs par défaut du dépôt et celles chargées depuis .env. Le dépôt peut démarrer en mode extrait tandis que cette machine utilise explicitement Ollama.

```
$requeteCours = @{
    question = "Quels documents pour devenir auto-entrepreneur ?"
    response_language = "fr"
} | ConvertTo-Json
Invoke-RestMethod -Method Post `
    -Uri http://127.0.0.1:8000/v1/answers `
    -ContentType "application/json; charset=utf-8" `
    -Body ([Text.Encoding]::UTF8.GetBytes($requeteCours))
```

Ce qu'il faut observer

Suis original, normalized et search_text. Vérifie detected, reply et method. Lis outcome et generation_failed avant de lire answer. Compare les citations avec les passages originaux, puis examine les timings. Une réponse différente de l'ancien rapport peut être normale si la revue a expiré, le modèle a changé ou le décodage a produit une autre sortie.

Enfin, ouvre pipeline.py et retrouve ces étapes dans le même ordre. Lis ensuite le test qui correspond à l'erreur qui t'intéresse. Cette lecture guidée est plus efficace que d'examiner tous les fichiers sans question précise.

Si l'API est arrêtée, une erreur de connexion ne dit rien sur la qualité du modèle. Si elle répond 503, commence par son motif de readiness. N'essaie pas une réingestion avec l'API ouverte sur le même Qdrant local.

Les prochaines améliorations

Transformer les limites observées en expériences

La prochaine étape doit répondre à un défaut mesuré. Ajouter un framework, un modèle plus gros ou une base différente n'est pas une amélioration en soi. Définis d'abord le critère de réussite et garde une baseline pour savoir si le changement a servi.

Chantier	Expérience proposée	Preuve de réussite
Darija générée	Comparer prompts/modèles sur questions gelées.	Fidélité et langue jugées, couverture conservée.
ASR Lmaana 2.4	Exécuter un clip puis un test tenu à part.	Transcriptions, WER/CER, mémoire et latence.
Recherche sémantique	Comparer lexical et Qwen embeddings.	Recall/MRR et abstentions sur le même jeu.
Corpus	Ajouter une source revue avec cas négatifs.	Extraction fidèle et réponses traçables.
Usage vocal	Ajouter correction de transcription.	Erreurs reconnues et récupérables par l'utilisateur.

Une expérience, une variable principale

Pour étudier l'embedding, conserve autant que possible les questions, les sources et le générateur. Pour étudier la génération, fournis exactement les mêmes preuves. Pour étudier l'ASR, évalue d'abord les transcriptions sans RAG. Une ablation enlève ou remplace un mécanisme, par exemple les alias, afin de mesurer son apport.

Les essais doivent prévoir un lot de développement pour ajuster les règles, un lot de validation pour choisir une configuration et un test final tenu à part. Ne réutilise pas continuellement le test final comme tableau de bord d'optimisation : il finirait par influencer toutes tes décisions.

Ce que LangChain ou LangGraph changeraient

Ces outils pourraient structurer des chaînes ou des graphes d'états, notamment avec boucles et outils. Le pipeline actuel est du Python explicite. Introduire un framework ne fournit pas automatiquement de meilleures sources, une meilleure Darija ou des métriques. La complexité ajoutée doit avoir un bénéfice concret de maintenance ou de comportement.

La cible est un assistant utile, pas une démonstration de noms de bibliothèques. Tu dois pouvoir expliquer chaque ajout par une erreur qu'il corrige et une mesure qui le confirme.

ANNEXE A · EXERCICES

Vérifier les bases

Essaie avant de lire les corrigés

1. Modèle ou moteur ?

Classe les éléments suivants : Qwen3-4B, Ollama, Lmaana 2.4, Qdrant, FastAPI. Pour chacun, donne son entrée et sa sortie principales dans le projet. Explique pourquoi « j'ai installé Ollama » ne signifie pas « mon assistant connaît les procédures à jour ».

2. Une question en Darija, une source française

La détection trouve ary, le meilleur passage est français et l'utilisateur n'a pas forcé de langue. Quelle langue cible le générateur ? Si le système est en mode extrait, quelle langue doit-il annoncer ? Si la rédaction Darija échoue deux fois, que doit comprendre l'utilisateur ?

3. Le document vient d'être téléchargé

Une brochure publiée plusieurs années auparavant est collectée aujourd'hui. Son hash est correct et son hôte officiel. Peut-elle soutenir une réponse actuelle ? Quelles informations manquent pour appliquer la politique du projet ?

4. Calcul de similarité

Avec $q=(1,0,1)$, $a=(1,0,0)$ et $b=(0,1,0)$, calcule les deux cosinus. Le score le plus élevé garantit-il que le passage répond à la question ? Que se passe-t-il si la question produit un vecteur nul ?

5. Budget de contexte

Exemple fictif avec comptage exact : fenêtre 4 096 tokens, instructions + question + template = 900, preuves = 2 500, sortie réservée = 900. Le budget est-il valide ? Combien faut-il retirer au minimum ? Pourquoi préférer retirer un passage entier à couper sa dernière condition ?

6. Mémoire approximative

Estime la taille brute de quatre milliards de poids à 16 bits puis à 4 bits, en Go décimaux. Explique pourquoi ce calcul ne permet pas de conclure qu'un modèle et un ASR tiendront ensemble dans 8 Go de VRAM.

Critère de réussite : tu dois pouvoir expliquer le raisonnement, pas seulement fournir un nombre. Les corrigés distinguent systématiquement le résultat mathématique et ce qu'il permet réellement de conclure.

ANNEXE A · EXERCICES

Diagnostiquer et évaluer

Des erreurs plausibles, pas seulement des cas faciles

7. Réduction CTC

Réduis la séquence [a, a, blanc, a, b, b, blanc]. Puis compare à [a, a, a, b, b]. Pourquoi faut-il fusionner les répétitions avant de supprimer les blancs ?

8. Calcul WER

Référence fictive : « je veux une photo ». Hypothèse : « je veux deux photos demain ». Utilise des tokens séparés par espaces et aucun rapprochement de singulier/pluriel. Donne S, D, I et le WER avec un alignement optimal.

9. Calcul de recherche

Trois questions ont leur premier passage pertinent aux rangs 1, 4 et absent. Calcule MRR@5 et Hit@3. Peux-tu calculer Recall@5 sans connaître tous les passages pertinents de chaque question ?

10. Citation valide, phrase fausse

Source fictive : « Déposer le dossier dans les 10 jours suivant l'inscription. » Réponse : « La carte est délivrée sous 10 jours. » L'identifiant de citation est valide et le nombre est présent. Pourquoi le contrôle de format et de nombres ne suffit-il pas ?

11. Une seconde requête pendant l'inférence

Le GPU traite une question, une deuxième arrive, puis la première dépasse le timeout HTTP. Quels statuts le projet peut-il retourner ? Pourquoi ne pas libérer simplement toutes les protections dès que le navigateur ferme la connexion ?

12. Le modèle change, pas les documents

Tu remplaces seulement le générateur par un autre modèle, en conservant l'encodeur et le corpus. Faut-il toujours reconstruire Qdrant ? Quelles informations doivent en revanche entrer dans la clé d'un résultat mis en cache et dans le rapport d'expérience ?

13. Une amélioration suspecte

Après avoir durci les garde-fous, le pourcentage de réponses acceptées et fidèles passe de 80 % à 95 %, mais seulement 10 questions sur 100 reçoivent désormais une réponse au lieu de 70. Est-ce une amélioration du produit ? Quels deux axes présenterais-tu ?

14. Lire le rapport local

Peut-on présenter les 10 issues attendues sur 14 scénarios comme une précision de 71 % en Darija ? Donne au moins trois raisons de refuser cette interprétation.

ANNEXE B · CORRIGÉS

Corrigés : les bases

Une bonne réponse précise aussi ses limites

1. Rôles

Qwen génère du texte à partir d'un contexte ; Ollama exécute un modèle et expose des appels HTTP ; Lmaana transforme l'audio en transcription ; Qdrant recherche des vecteurs et renvoie leurs métadonnées ; FastAPI reçoit des requêtes et expose les opérations du pipeline. Installer un moteur n'installe pas une politique de corpus actuel.

2. Langue et résultat

En mode génératif, la cible est ary. En mode extrait, le français reste du texte source et `answer_language` doit être source. Après deux sorties invalides, `generation_failed=true` signale une réponse non validée ; il ne faut pas prétendre que la source française a été correctement traduite.

3. Fraîcheur

Non, ces éléments ne suffisent pas. Il faut une revue de l'usage actuel, datée, liée au hash et au périmètre approprié, avec une échéance encore valide et des justificatifs. Un téléchargement récent donne seulement une date de collecte ; le hash établit l'identité des octets.

4. Cosinus

$q \cdot a = 1$, $\|q\| = \text{racine}(2)$, $\|a\| = 1$: $\cos(q, a) = 1/\text{racine}(2)$, environ 0,707. $q \cdot b = 0$ donc $\cos(q, b) = 0$. La proximité ne prouve pas la réponse. Le cosinus du vecteur nul est indéfini mathématiquement ; l'implémentation doit gérer ce cas et ne pas produire une division par zéro.

5. Contexte

$900 + 2\,500 + 900 = 4\,300$: le budget dépasse la fenêtre de 204 tokens. Il faut économiser au moins 204 tokens, et souvent davantage si l'on retire une unité documentaire entière. Une coupe mécanique peut conserver une règle en supprimant son exception. Dans le vrai adaptateur Ollama, le précontrôle est conservateur en octets et non exact comme ici.

6. Poids et mémoire

$4 \text{ milliards} \times 16 / 8 = 8 \text{ milliards d'octets}$, soit 8 Go décimaux bruts. À 4 bits : 2 Go bruts. Les formats quantifiés ajoutent des informations ; le runtime utilise aussi cache KV, activations et buffers. L'ASR, l'affichage et les transferts ajoutent leurs besoins. Il faut mesurer les pics, pas seulement additionner les tailles de fichiers.

ANNEXE B · CORRIGÉS

Corrigés : les diagnostics

Regarder la cause, le protocole et l'expérience utilisateur

7. CTC

La première séquence devient [a, blanc, a, b, blanc], puis « aab ». La seconde devient « ab ». Supprimer d'abord les blancs ferait se toucher deux a qui doivent rester distincts ; une fusion ultérieure supprimerait alors une répétition légitime.

8. WER

L'alignement conserve « je veux », remplace « une » par « deux », remplace « photo » par « photos » et insère « demain ». S=2, D=0, I=1, N=4. WER=3/4=75 %. Une autre politique de normalisation pourrait changer le score ; elle doit être annoncée avant comparaison.

9. Recherche

MRR@5=(1 + 1/4 + 0)/3=5/12, environ 0,417. Seule la première question a un pertinent dans le top-3 : Hit@3=1/3. Le rappel nécessite le nombre total de pertinents et le nombre récupéré ; le rang du premier ne suffit pas.

10. Le mauvais événement

La source parle du dépôt après inscription, la réponse de délivrance de carte. Le nombre est conservé mais son événement d'application est changé. Une vérification sémantique doit contrôler qui fait quoi, dans quel délai, à partir de quel point de départ. Une regex sur les chiffres ne couvre pas cette relation.

11. Concurrence

La seconde requête peut recevoir 429. La première peut recevoir 504 après timeout ; les suivantes rencontrent une indisponibilité 503 tant que l'API n'est pas rétablie. La fin de l'attente HTTP ne garantit pas l'arrêt du calcul côté runtime : autoriser immédiatement un nouveau calcul pourrait provoquer une surcharge.

12. Changement de générateur

Pas de réingestion nécessaire pour ce seul changement. Il faut en revanche identifier le nouveau générateur, ses poids/digest, son template et ses réglages, et invalider les réponses mises en cache pour l'ancien profil. Le corpus et les politiques restent également dans l'identité du résultat.

13 et 14. Ne pas masquer les échecs

Présente qualité conditionnelle et couverture ensemble : 95 % parmi seulement dix réponses n'est pas équivalent à 80 % parmi soixante-dix. Le rapport local mélange langues, abstentions et clarification ; il est petit et non représentatif ; quatre réponses souhaitées ont échoué. Il ne mesure donc pas une précision générale Darija.

ANNEXE C · GLOSSAIRE

Le vocabulaire à retenir

Modèles, langue et documents

Terme	Définition dans ce cours
ASR	Reconnaissance automatique de la parole : audio vers texte.
TTS	Synthèse vocale : texte vers audio.
VAD	Détection des segments contenant probablement de la parole.
CTC	Objectif d'apprentissage de séquences avec alignements non fournis.
Token	Unité manipulée par un tokenizer ; pas nécessairement un mot.
Tokenizer	Convertit texte et identifiants de tokens selon une convention.
Poids / paramètres	Nombres appris qui définissent la fonction du modèle.
Inférence	Exécution d'un modèle pour obtenir une sortie.
Fine-tuning	Adaptation des poids avec un entraînement supplémentaire.
Quantification	Représentation numérique plus compacte, avec perte possible.
GGUF	Format de stockage de modèles et métadonnées ; pas un modèle en soi.
Runtime	Logiciel qui charge et exécute le modèle.
Contexte	Séquence disponible au modèle pour la génération courante.
Cache KV	Mémoire des clés/valeurs d'attention des tokens déjà traités.
Code-switching	Alternance de langues au sein d'un échange ou d'une phrase.
Arabizi	Écriture latine de l'arabe/Darija, parfois avec chiffres.
Normalisation	Transformation contrôlée qui réduit des variations superficielles.
Intention	Type d'information recherché : documents, frais, délai, etc.
Provenance	Origine et traçabilité d'une donnée ou d'un passage.
Chunk	Unité documentaire découpée pour la recherche et le contexte.
Overlap	Contenu partagé entre deux chunks voisins.
SHA-256	Empreinte de contenu ; contrôle d'intégrité, pas certificat de vérité.

Deux mots proches ne sont pas toujours interchangeables : contexte n'est pas mémoire permanente ; modèle n'est pas moteur ; transcription n'est pas traduction.

ANNEXE C · GLOSSAIRE

Le vocabulaire à retenir

Recherche, logiciel et évaluation

Terme	Définition dans ce cours
RAG	Recherche de preuves externes suivie d'une génération contextualisée.
Embedding	Représentation vectorielle d'un objet ; sa qualité dépend de sa construction.
Cosinus	Mesure de proximité de direction entre vecteurs non nuls.
Top-k	Les k premiers éléments d'un classement.
Reranker	Composant qui reclasse une sélection préalable de candidats.
Payload	Métadonnées associées à un point vectoriel.
Manifeste	Description structurée d'une collection de fichiers ou d'artefacts.
Digest / révision	Identité précise d'un artefact ou état versionné.
Contrat	Forme et contraintes attendues pour les entrées/sorties.
JSON Schema	Description des champs, types et valeurs autorisés en JSON.
API / endpoint	Interface entre programmes / opération adressable.
Loopback	Adresse qui renvoie vers la machine locale.
Readiness	Disponibilité technique selon les préconditions du service.
Timeout	Limite d'attente ; pas forcément annulation distante.
Atomicité	Une opération visible comme un tout, pas un état partiel.
Mock	Comportement simulé utilisé pour tester un contrat ou une branche.
WER / CER	Taux d'erreurs d'édition au niveau mots / caractères.
Recall@K	Part des éléments pertinents retrouvés parmi les K premiers.
MRR	Moyenne de l'inverse du rang du premier résultat pertinent.
Fidélité	Degré de soutien des affirmations par les preuves fournies.
Couverture	Part des questions auxquelles le système répond selon une définition donnée.
Latence / débit	Temps par requête / volume traité par unité de temps.
Ablation	Expérience retirant ou remplaçant un composant pour mesurer son apport.

Le code du projet utilise parfois une définition opérationnelle plus étroite que le mot général. Toujours lire le contrat et la manière de mesurer avant d'interpréter un chiffre.

ANNEXE D · SOURCES

Références et traçabilité

Sources primaires consultées le 3 octobre 2026

R1. Lewis et al. - Retrieval-Augmented Generation (2020)

Fondement du couplage entre mémoire paramétrique et recherche externe.

R2. Vaswani et al. - Attention Is All You Need (2017)

Article fondateur du Transformer et du mécanisme d'attention.

R3. Qwen - Qwen3-4B-GGUF, fiche officielle

Architecture, formats quantifiés et usage du modèle génératif.

R4. Ollama - API de génération de chat

Messages, paramètres de génération et forme des réponses du runtime.

R5. Ollama - Structured Outputs

Contraindre la forme de sortie avec un schéma JSON.

R6. Lmaana - lmaana-2.4, fiche officielle

Cible ASR, runtime fairseq2, CTC, accès et protocole publié.

R7. Graves et al. - Connectionist Temporal Classification (2006)

Objectif CTC, alignements et réduction des séquences.

R8. Whisper-large-v3-turbo - fiche du modèle

Baseline ASR prévue ; pas de résultat comparatif propre au projet.

R9. Qdrant - Collections

Points, dimensions, métriques et conventions de collection.

R10. Qwen3-Embedding-0.6B - fiche officielle

Encodeur sémantique optionnel et conventions de requête.

Les exemples et exercices sont des constructions pédagogiques originales. Les valeurs propres à cette machine viennent des fichiers locaux, pas des benchmarks publiés par les auteurs des modèles.

ANNEXE D · SOURCES

Références et traçabilité

Documentation technique et preuves locales

R11. FastAPI - Request Body

Corps de requêtes et modèles de données.

R12. Pydantic - Models

Validation et représentation des données typées.

R13. Streamlit - Caching overview

Réutilisation des calculs et gestion du cache.

R14. OWASP - SSRF Prevention Cheat Sheet

Défense en profondeur pour les requêtes serveur vers des URLs.

R15. Ollama - FAQ officielle

Fonctionnement local, réseau et désactivation du cloud.

R16. Python - venv

Environnements Python isolés.

R17. uv - Locking and syncing

Verrouillage et synchronisation des dépendances.

R18. Manning, Raghavan, Schütze - évaluation de la recherche

Interprétation des classements et nécessité des annotations.

R19. FAISS - documentation officielle

Bibliothèque de recherche vectorielle évoquée comme alternative.

R20. Pro Git - About Version Control

Principes du contrôle de versions.

Preuves locales : README.md ; docs/lmaana-2.4-integration.md ; docs/source-quality.md ; code sous src/lmaana_assistant/ ; tests/ ; outputs/Lmaana-Ollama-Checks.json. Les captures et rapports décrivent des exécutions antérieures datées, pas une certification de production.